SEQ2SEQ
Encoder-Decoder

Why Encoder-Decoder ?

- Despite being very powerful, DNNs cannot map sequences to sequences. They can only map vectors to vectors.
- Learning to map sequences to sequence is very important in tasks like
  Machine translation
  Speech recognition
  Image caption generation and many other tasks.
- But we know that RNNs can work with sequences then the problem ?
- In this u have sequence of inputs and sequence of outputs but both of them have the same alignment. So they have one-to-One correspondence between them. And because of that they have long term dependency problems due to exploding and vanishing gradient problem.

Why Encoder-Decoder ?

- LSTMs are similar to RNNs but here we are less likely to get vanishing gradient problem
- Key difference between LSTMs and RNNs ->
  RNN -> overwrites the hidden state
  LSTM -> Adds to the hidden state
- This small difference is very significant because final hidden state is sum of many little Δ and because of this
- Every Δ gets a gradient and doesnt vanish because of the sum.
- And since it is a sum how does it know about order ?
- Because LSTM decides what to add based on the sequence

Inside Encoder

- LSTM cell is there
- After sending sequence we get final cell state(ct) and hidden state(ht). That is the final representation of the sentence (contect vector)
- Then we pass this context vector to the decoder.

Inside Decoder

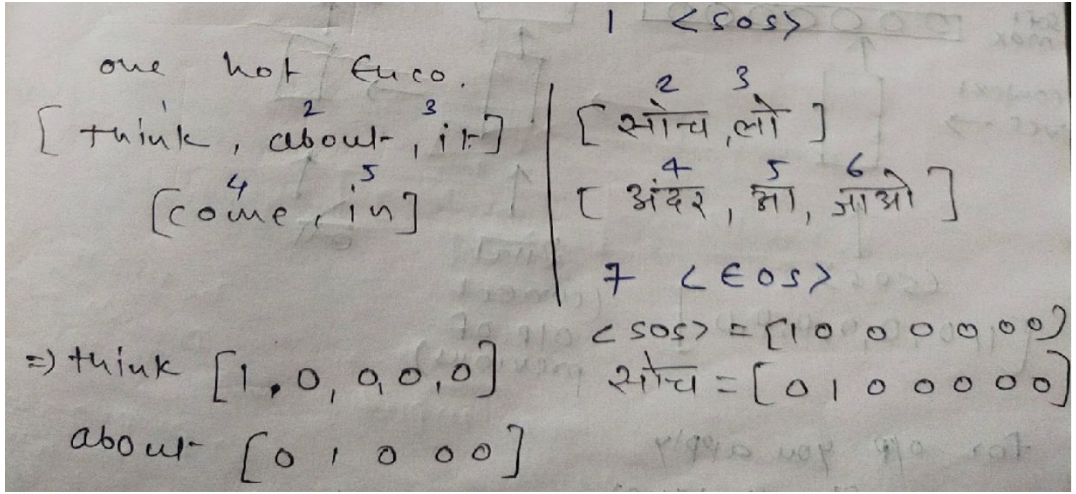
- Contains LSTM
- It will produce the o/p at each time step
- Initial state of LSTM will be same as final state of encoder (context vector)
- At time step = 1
  You send a special symbol for denoting that this is the start of the sentence (<sos>).
  Basically this tells decoder to start producing the o/p.
- At time step = 2
  You pass the o/p of t=1 as i/p along with the internal states.
  Continue this till we get <eos> (end of sentence token) in the o/p.

Example.....

- Taking the example of machine translation
- For simplicity using one hot encoding but we generally use embeddings of the words
- In case anyone is wondering how to generate embeddings of the words of a different language than english ?
- The process of generating embeddings for words in different languages doesn't fundamentally differ from English. The system does not "understand" the language itself but instead learns patterns and relationships between tokens (words, subwords, or characters) through numerical representations

Example…….

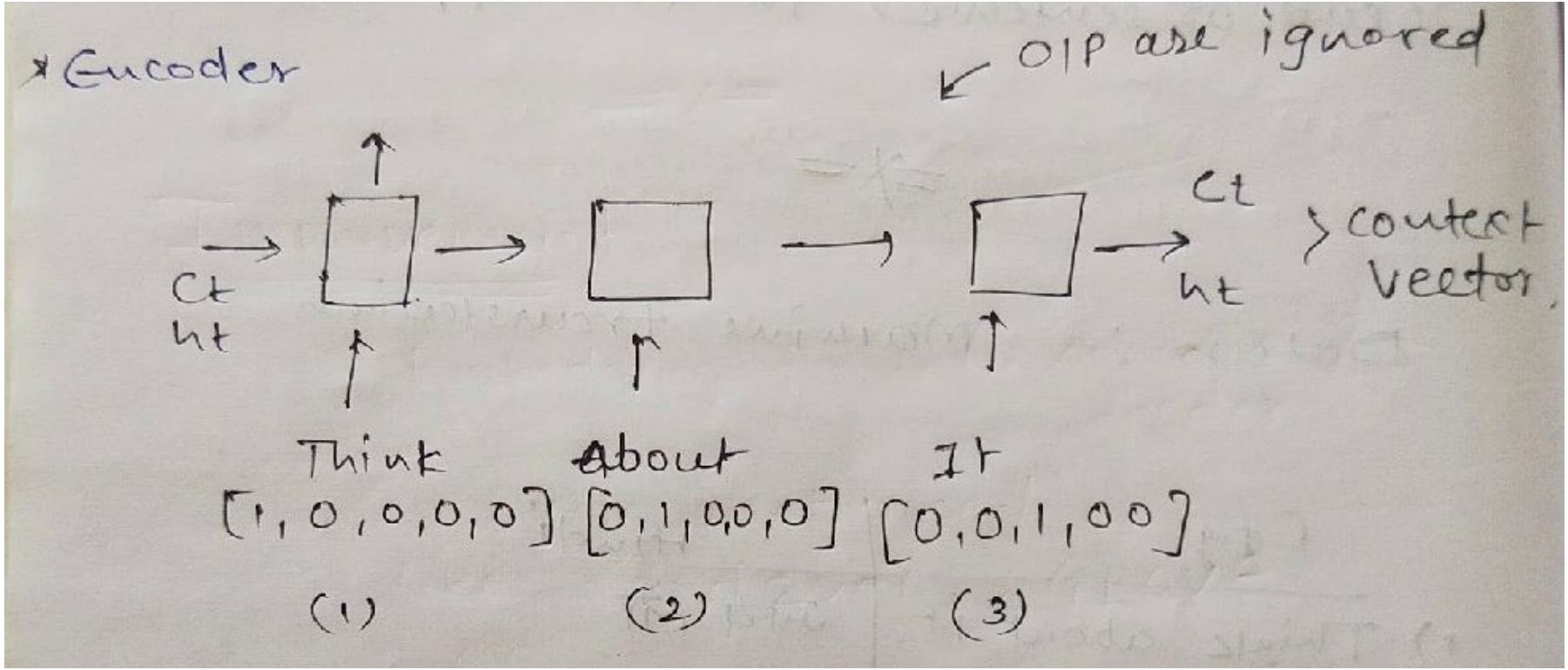
- Consider english to hindi
- Here we will apply OHE But the number of words in the vocabulary of the output will differ because we have to add start and end of sentence special tokens (sos, eos)



Encoder: Training

- Here we will train the model on our supervised data
- In encoder which is comprised of LSTM we pass our one hot encoded inputs at each time stamp
- And at the end we get context vector with the help of cell state and hidden state (we concat ct and ht)

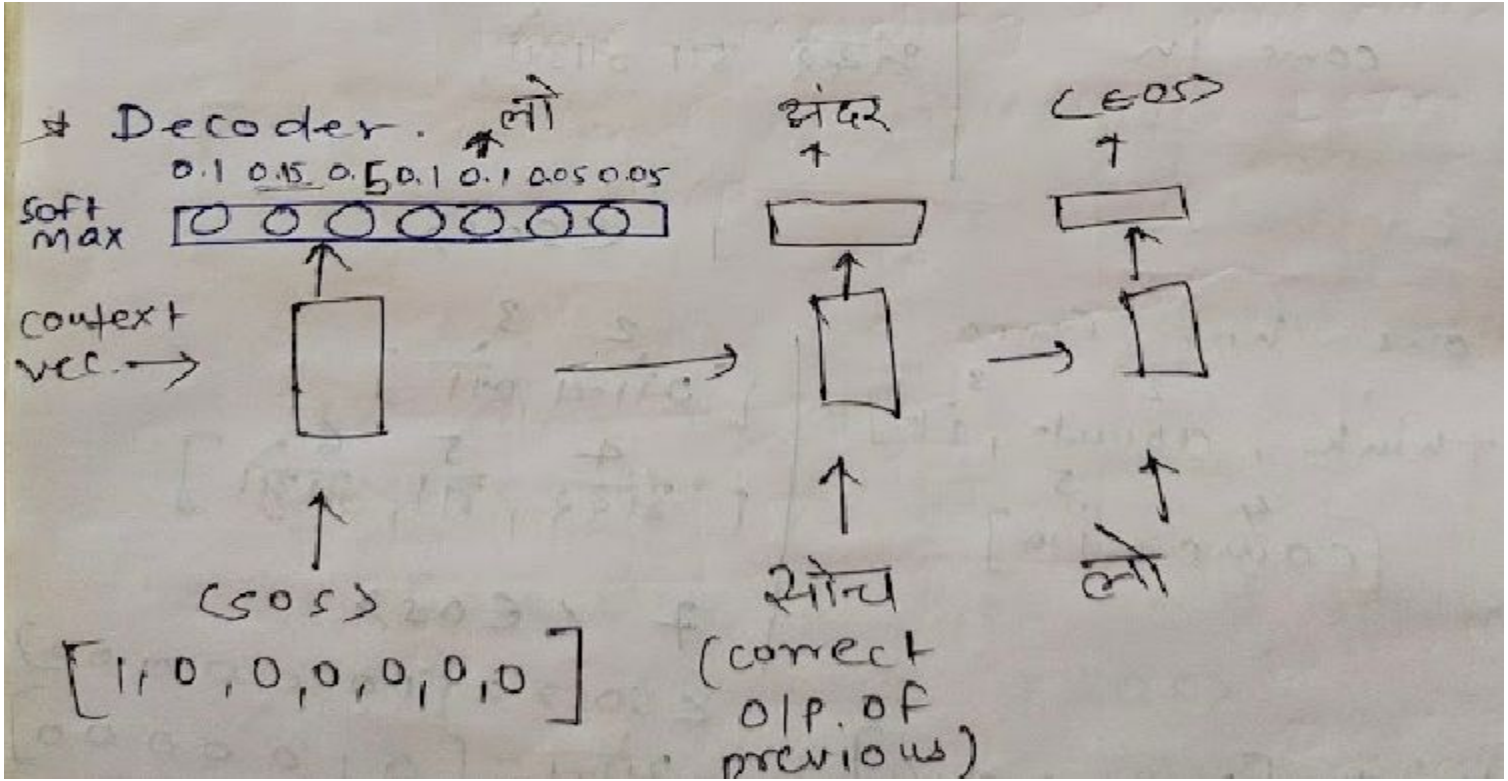
Encoder: Training



Decoder: Training

- Here decoder part is a bit tricky: It behaves differently during training than during inferencing(prediction).
- During training
- Decoder at each time step it is possible that you may get wrong output and we know that the output at t=1 will be used as input at t=2, So it affect it negatively if model predicts wrong at any time step.
- So instead of sending the o/p of previous time step we will be sending the correct output manually to the current timestamp
- This concept is called **teacher forcing.**
- **So after one forward propagation you calculate the loss and try to minimize it through stochastic gradient descent.**

Decoder: Training



Improvements we can make

- Instead of OHE, use embeddings for both encoder and decoder.
- Instead of single layer LSTM use deep LSTM
- In the encoder you can reverse the input.
- In the seq2seq paper they also found that reversing the i/p actually increased the accuracy
- They were not really sure about why this was the case but one possible reason is that it propagates the gradients a bit better.